

Your RAG Code, Explained Simply

Same project as before — this time broken down line by line, in plain language

1. What is this program actually doing?

Think of it like this: you have a text file with some information in it. You chop that file into small pieces, turn each piece into a set of numbers a computer can compare (an "embedding"), and store those numbers in a searchable database called Pinecone.

Later, when someone asks a question, the program does not just guess the answer. It first goes and finds the 2-3 pieces of your original text that seem most related to the question, hands those pieces to the AI along with the question, and says: "answer using only this information." That is the whole idea behind RAG — Retrieval-Augmented Generation. Retrieval means fetching the right pieces of text. Generation means the AI writing the final answer.

There are two files doing this job: ingestion.py, which prepares and stores your data, and main.py, which answers questions using that stored data.

2. ingestion.py — line by line

```
loader = TextLoader("data/mediumblog1.txt", encoding="utf-8")
document = loader.load()
```

This opens your text file and reads it into memory as a document object. The encoding="utf-8" part just tells Python how to interpret the bytes in the file as text — this matters because if a file is saved in a different format (like UTF-16), the text comes out scrambled, which is exactly the bug we fixed earlier.

```
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
texts = text_splitter.split_documents(document)
```

AI models and search systems work better with small pieces of text rather than one giant block. This chops your document into pieces of about 1000 characters each. chunk_overlap=0 means the pieces do not repeat any text between them — each chunk starts exactly where the last one ended.

```
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

This loads a small AI model whose only job is to turn a piece of text into a list of numbers (a vector) that captures its meaning. Two pieces of text about similar topics end up with similar-looking number lists. This is what makes "search by meaning" possible, instead of just matching exact keywords.

```
pc = Pinecone(api_key=os.environ.get("PINECONE_API_KEY"))
index = pc.Index(os.environ.get("INDEX_NAME"))
index.delete(delete_all=True)
```

This connects to your Pinecone account (using a secret key stored in your .env file) and opens a specific "index" — think of an index as a labelled drawer where your data lives. delete_all=True empties that drawer first, so you never end up with duplicate or stale data mixed in with a fresh run.

```
PineconeVectorStore.from_documents(
    texts,
```

```

    embeddings,
    index_name=os.environ.get("INDEX_NAME")
)

```

This is the actual ingestion step. For every chunk of text, it runs the embedding model to get its number list, and saves both the original text and its number list into Pinecone. After this line finishes, your data is searchable.

3. main.py — line by line

```

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
llm = ChatGroq(model="llama-3.3-70b-versatile")

```

Same embedding model as before — it has to be the same one used during ingestion, otherwise the number lists would not be comparable. llm is the actual "brain" that will write the final answer, running on Groq's fast servers.

```

vectorstore = PineconeVectorStore(
    index_name=os.environ.get("INDEX_NAME"),
    embedding=embeddings
)

```

This reconnects to the same Pinecone drawer you filled during ingestion, so this file can search through it.

```

retriever = vectorstore.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"k": 3, "score_threshold": 0.5}
)

```

This turns your storage drawer into a "retriever" — something you can hand a question to, and it hands back the most related chunks. k: 3 means "give me at most 3 chunks." score_threshold: 0.5 is the important safety net — it means "only give me chunks that are at least reasonably related; if nothing clears that bar, give me nothing." Without this threshold, the retriever always returns 3 chunks no matter how unrelated they are, which is what let the AI hallucinate answers to unrelated questions earlier.

```

prompt_template = ChatPromptTemplate.from_template("""
Answer the question based only on the following context.
If the context does not contain enough information, respond with:
"I don't have enough information..."
Context: {context}
Question: {question}
""")

```

This is the instruction sheet handed to the AI every single time. {context} and {question} are blanks that get filled in later with the retrieved text and the user's actual question. Telling the AI explicitly what to say when it does not know the answer is what stops it from making things up.

```

def format_docs(docs):
    return "\n\n".join([doc.page_content for doc in docs])

```

The retriever hands back a list of chunk objects, not plain text. This function just pulls the plain text out of each chunk and glues them together with blank lines in between, so it can be dropped into the {context} blank.

4. Version 1 — Without LCEL (the manual way)

This is the version you had first. It works by doing every step yourself, one after another, as separate lines of code.

```
def retrieval_chain_without_lcel(query: str):
    docs = retriever.invoke(query)
    context = format_docs(docs)
    messages = prompt_template.format_messages(
        context=context, question=query
    )
    response = llm.invoke(messages)
    return response.content
```

Step by step, in plain words:

- `docs = retriever.invoke(query)` — go fetch the related chunks for this question.
- `context = format_docs(docs)` — turn those chunks into one plain text block.
- `messages = prompt_template.format_messages(...)` — fill in the instruction sheet with the context and the question.
- `response = llm.invoke(messages)` — send the filled-in instructions to the AI and wait for its reply.
- `return response.content` — hand back just the text of the answer, not the extra wrapping around it.

This works completely fine. The downside is that you wrote every single step by hand, in a fixed order. If you wanted this to stream the answer word-by-word, or run several questions at once, or run it without blocking your program, you would need to write extra code for each of those separately.

5. Version 2 — With LCEL (the pipeline way)

LCEL stands for LangChain Expression Language. Instead of writing steps one after another, you describe the same steps as a pipeline, connected with the `|` symbol — the same idea as piping commands together in a terminal, where the output of one step automatically becomes the input of the next.

```
def create_retrieval_chain_with_lcel():
    retrieval_chain = (
        RunnablePassthrough.assign(
            context=itemgetter("question") | retriever | format_docs
        )
        | prompt_template
        | llm
        | StrOutputParser()
    )
    return retrieval_chain
```

Reading this piece by piece:

- `itemgetter("question")` — when the chain is later given something like `{"question": "..."}`, this just pulls out the question text from it.
- `| retriever` — takes that question text and feeds it straight into the retriever, which fetches the related chunks.
- `| format_docs` — takes those chunks and turns them into one plain text block, exactly like before.
- `RunnablePassthrough.assign(context=...)` — this whole pipeline result gets attached as a new field called `context`, while keeping the original question alongside it. So after this line, you have both question and context sitting together.

- | prompt_template — both of those get poured into the instruction sheet, filling in its blanks automatically.
- | llm — the filled-in instructions get sent to the AI.
- | StrOutputParser() — the AI's reply gets unwrapped down to plain text automatically, so you do not need to type .content yourself.

You call it like this:

```
chain_with_lcel = create_retrieval_chain_with_lcel()
result_with_lcel = chain_with_lcel.invoke({"question": query})
```

Notice the result is already a plain string — no .content needed, because StrOutputParser() already handled that unwrapping inside the pipeline.

6. Which one is better, and why

Aspect	Without LCEL (manual)	With LCEL (pipeline)
How steps connect	Written one line at a time, in a fixed order you control fully	Connected automatically with the operator
Amount of code	More lines, more manual wiring	Fewer lines for the same result
Streaming answers word-by-word	Not built in — you would have to add it yourself	Built in — chain.stream() just works
Running without blocking (async)	Not built in	Built in — chain.ainvoke()
Running many questions at once	Not built in	Built in — chain.batch()
Easiest to debug when learning	Yes — every step is a visible, separate line you can print	Slightly harder to trace at first, since steps are chained together
Best for	Learning exactly how retrieval and prompting work under the hood	Real, production-style use where speed and less code matter

The honest answer: neither one is "wrong." They do the exact same job and give the exact same answer.

- Use the version without LCEL while you are learning — it forces you to see every single step happening, which makes it much easier to understand what RAG actually is.
- Use the version with LCEL once you understand the steps and want to build something real — it gives you streaming, async, and batching for free, with less code to maintain, which is exactly why LangChain recommends it for production use.

7. The LangChain Agent code — same idea, explained simply

The second piece of code you shared is a different project: instead of just answering from a fixed text file, it builds an AI "agent" that can search the internet on its own and hand back a neatly structured answer.

What it is actually doing

You are hiring an AI assistant, giving it exactly one tool — "search the internet" — and two rules: do not search more than 3 times, and instead of rambling in a paragraph, fill in a fixed form with the answer and the source links.

BaseModel and Field — the form template

```
class Source(BaseModel):
    url: str = Field(description="The URL of the source")
```

BaseModel is a form template. This creates a form called Source with exactly one box on it, url, and that box must contain text. If you tried to put a number in that box instead, you would get an error immediately instead of a silent bug later. Field() lets you attach a note to that box — and that note is not just for humans reading the code, it gets shown directly to the AI, so a clearer description genuinely helps the AI fill in that box correctly.

```
class AgentResponse(BaseModel):
    answer: str = Field(description="The answer generated by the agent")
    sources: List[Source] = Field(default_factory=list, description="...")
```

This is the final form the AI must fill in. answer must be text. sources must be a list, and every item inside that list must itself be a proper Source form, not just plain text. default_factory=list means "if nobody supplies any sources, start with an empty list" — written this way specifically to avoid a well-known Python trap where a single [] written as a default gets shared and accidentally reused across every response.

Version 1 — building the search tool by hand

```
tavily = TavilyClient()

@tool
def search(query: str) -> str:
    """Tool that searches over internet"""
    print(f"Searching for: {query}")
    return tavily.search(query=query)
```

- TavilyClient() — makes one reusable connection to the Tavily search service.
- @tool — the important line. It looks at the function below it — its name and its explanation — and turns it into something the AI is allowed to call by name. The AI never sees the Python code itself, only the name and the explanation.
- The text inside the triple quotes is not just a comment — it becomes the tool's description, which is literally what the AI reads to decide whether it should use this tool right now.
- return tavily.search(query=query) — actually runs the search and hands the results back to the AI.

Version 2 — using a tool someone already built

```
tools = [TavilySearch()]
```

One line does the same job as all of Version 1's code. TavilySearch() already comes with its own name, its own explanation, and already knows how to call the Tavily API. The only catch: the AI's instructions say it only has a tool called "search," but this ready-made tool is actually named tavily_search. It still works because the AI only has one tool to choose from anyway, but it is a small mismatch worth knowing about.

create_agent — building the loop

```
agent = create_agent(
    model=llm,
    tools=tools,
    response_format=AgentResponse
)
```

You hand over the brain (model), the toolbox (tools), and the exact form it must fill in at the end (response_format). In return you get back a ready-to-run loop that: sends messages to the AI, checks if it asked to use a tool and runs it if so, feeds the result back, repeats, and once the AI stops asking for tools, makes it fill in the final form. You never write this loop by hand.

Version 1 vs Version 2 — side by side

Aspect	Version 1 (by hand)	Version 2 (ready-made)
Lines of code needed	About 13 lines	1 line
Name the AI sees for the tool	"search" — matches the instructions exactly	"tavily_search" — slight mismatch with the instructions
Control you have	Full — rename it, reshape results, add logging	Limited to what the constructor allows
Best for	Learning how tools work under the hood	Shipping something quickly, with less to maintain

8. The one-paragraph takeaway

Across both projects, the pattern is the same: there is always a "do it by hand" version that shows you exactly how something works underneath, and a "ready-made" version that gets you the same result faster with less to maintain. Neither is more correct. Use the manual version — without LCEL, or Version 1 with @tool — when you are learning or need full control. Use the ready-made version — with LCEL, or Version 2 with TavilySearch — once you understand the mechanics and just want it working, fast.